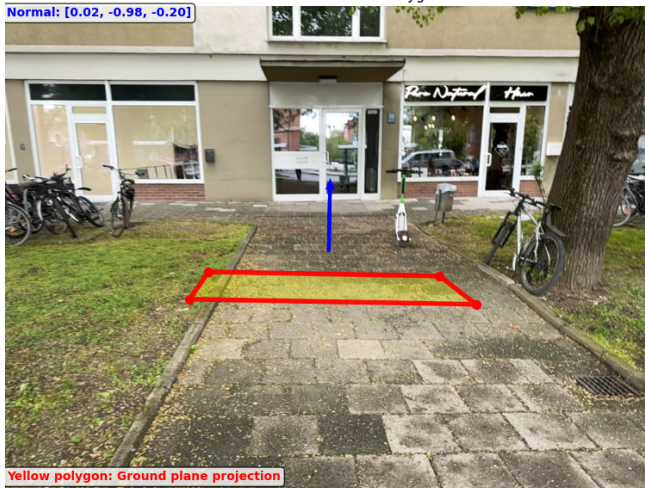


3D Spatial Scene Description for and Guidance for Visually Impaired People

Jan Duchscherer
j.duchscherer@hm.edu
University of Applied Sciences Munich
Munich, Bavaria, Germany

Lukas Ederer
ederer0@hm.edu
University of Applied Sciences Munich
Munich, Bavaria, Germany



(a) RANSAC ground plane detection for height estimation.



(b) Segmentations of objects relevant for navigating to the entrance of the hair salon.

Figure 1: Overview of our 3D spatial understanding system: (a) Ground plane detection using RANSAC algorithm with object detection and spatial relationships, and (b) Interactive guidance system providing spatial descriptions and navigation assistance.

Abstract

This paper presents a 3D spatial understanding system designed to provide navigation assistance for visually impaired individuals. Our approach combines computer vision techniques with large language models to create an interactive audio guidance system that can describe spatial relationships and provide real-time navigation instructions. The system utilizes RANSAC-based ground plane detection, semantic segmentation, and spatial reasoning to understand complex 3D environments and communicate spatial information effectively [3].

Keywords

3D Spatial Understanding, Assistive Technology, Visually Impaired, Gemini, Large Language Models (LLMs), Interactive Audio Guidance, Semantic Segmentation, Mobile Navigation, Human-Computer Interaction (HCI), Obstacle Avoidance

ACM Reference Format:

Jan Duchscherer and Lukas Ederer. 2025. 3D Spatial Scene Description for and Guidance for Visually Impaired People. In *Proceedings of Deep Learning in Visual Computing - Summer Semester 2025 (DLVC SS 2025)*. ACM, New York, NY, USA, 13 pages.

1 Introduction

It is not always easy to navigate a city environment, especially for people with visual impairments. Often times there are obstacles in the way that can be hard to detect, or there is no clear path to follow. Using common navigation systems or apps that use 2D data or gps is not always sufficient to provide the necessary information for navigation. Often times it is necessary to have a 3D understanding of the environment in order to navigate around safely. This is where the idea of a 3D spatial scene description system comes into play. The goal of this project is to create a system that can help users navigate through their environment by providing spatial guidance. The app combines known VLM capabilities with

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DLVC SS 2025, Munich, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

3D data to provide spatial information. It should help the user by giving him information about the location, height and distance of objects around him. There should also be a way of getting further navigation assistance by providing a way to ask questions about the environment.

1.1 Research Question

After analyzing the current state of the field and the above mentioned goals, the following research question was formulated:

How can a 3D spatial scene description system effectively translate visual and depth information into guidance to improve navigation of blind and visually impaired individuals.

1.2 Structure of the Report

At first the report will give an overview of the background and related work that is relevant to the project. This includes software and algorithms that were discovered during the research phase and used further on. After that a general overview over the concepts and the system architecture will be provided. In order to evaluate the approach that is shown in the concept section, multiple tests were conducted. These will be presented in the evaluation section and can give an idea of the performance and accuracy of the system. Afterwards in the discussion section the report will focus on possible limitations that were found during the evaluation. At last a conclusion with a summary of the project and a outlook on future work will be presented.

2 Background

2.1 Robust Plane Detection with RANSAC

A reliable estimate of the ground plane is essential for converting per-pixel depth values into metric *heights*. We adopt the classic *Random Sample Consensus* (RANSAC) paradigm. Given an unstructured point cloud $\mathbf{P} = \{\mathbf{p}_i^c\}$, RANSAC iteratively (1) samples three non-collinear points, (2) fits a candidate plane $\hat{\mathbf{n}}^T \mathbf{x} + d = 0$, (3) counts inliers $\|\hat{\mathbf{n}}^T \mathbf{p}_i + d\| < \epsilon$, and (4) retains the model with maximal consensus. The pseudo-code in algorithm 4 runs in $\mathcal{O}(N_{\text{iter}}|\mathcal{P}|)[3, 10]$.

2.2 Gemini’s Image Understanding Capabilities

Google’s Gemini models are *natively multimodal* and offer powerful image understanding capabilities, including *zero-shot* semantic segmentation, as well as axis-aligned bounding box (AABB) and oriented bounding box (OBB) detection. The OBBs are provided as metric center points, axis-aligned dimensions, and euler angles, meaning that Gemini is already capable of quantitative reasoning about metric spatial relationships. However, since it only sees the RGB images, these outputs are not of sufficient trustworthiness for our use case, as they are not based on *true* depth estimates, but rather on the RGB image content [7].

In our work we utilized models of the Gemini 2.0 family, which are the most capable models in terms of OBB detection, while they lack the capability to provide semantic segmentation masks, which was introduced in the Gemini 2.5 family. Gemini 2.5 Pro and Flash both offer native *reasoning* capabilities to allow for more complex

spatial reasoning tasks, such as *pointing* [7], or identifying a destination in the scene and identifying a feasible path together with all obstacles and landmarks along the way. The latest Pro model is optimized for *enhanced thinking* and *multimodal understanding* and hence appears to be the most suitable model for our use case. The Google GenAI SDK supports structure outputs [8], which allows us to define a schema for the expected output format, which is essential for our use case, since complex data structures, such as segmentation masks, need to be parsed.

2.3 Gemini Live API

Real-time conversations leverage the **Live API** (preview) which streams responses with latencies < 100 ms when configured for text [6]. Only one response modality (Text or Audio) is allowed per session; audio-only sessions last 15 min, mixed Audio/Video sessions 2 min. Voice-Activity Detection (VAD) cancels generation on user interruption, and a `thinking_budget` allows the Gemini 2.5 Flash Live model to perform reasoning before emitting a response.

The Live API model supports *function calling*, which enables it to be employed as a *spatial guidance agent* that can provide *qualitative* scene descriptions and use *external tools* to answer *quantitative* queries.

3 Related Work

As discussed in previous studies [4], spatial understanding for assistive technology is an active research area.

3.1 Spatial Understanding Systems

3.1.1 SpatialBot. SpatialBot is a VLM designed for spatial tasks, by incorporating depth images and data into its outputs. It is trained on a hierarchy of spatial tasks, which are categorized as low, middle and high-level.

- **Low-level tasks:** Direct querying of depth values from a depth map and exact information of these on set points
- **Middle-level tasks:** Comparing distances between objects, and calculating the relative position of objects in a scene.
- **High-level tasks:** Understanding spatial relationships between objects and doing things like counting objects in a scene.

It has been successfully deployed in robotic systems, where it can be used to translate spatial understanding into physical actions. This highlights the robustness of its capabilities which would help in guidance systems, where it is important to get correct spatial information to the user.

Another feature that is highlighted in the paper is the ability to distinguish between real and printed objects by using the spatial depth information provided by the depth images. This can also help in navigation where detecting hazards is important, with also means the system needs to be able to decide if an object is a poster or a real thread.

Internally the SpatialBot framework uses a fine-tuned VLM with a curated RGB-D dataset. It tries to specifically address the challenge of providing consistent and accurate information for robotic systems. As seen later in this paper, we also used a lot of the same

concepts in mapping depth values and providing spatial information also seen here.[2]

3.1.2 Video-3D LLM. Video-3D LLM is a model that enhances Multimodal Large Language Models (MLLMs) with sophisticated 3D spatial understanding capabilities. This tackles the challenge, that most MLLMs are only trained on 2D data with limits there capabilities in spatial reasoning. It is treating 2D scenes as dynamic videos and incorporating 3D data into the video representation. This allows a more accurate understanding and goes in line with the goal of using a smartphone video stream as an input for our system.

- **Position-Aware Video Representation:** It is not using the traditional approach of converting 3D scenes into point clouds. Instead it directly processes video frames along with their corresponding 3D global coordinates. It does this by back-projecting pixel from the depth image and then encoding and injecting the visual features to give the option of getting spatial information.
- **Efficient Frame Sampling:** To handle larger video sequences, it uses a sampling strategy where it only uses the minimum number of frames needed to get all the spatial information.
- **Multi-Task Generalist Model:** As a single model it can handle task like, describing objects, locating objects and answering questions about objects that require spatial reasoning.

Even though it might seem like a perfect fit for our usecase it still comes with some limitations. For example, it is trained on indoor scenes and would probably not be able to handle outdoor scenes as well. Its compute time for processing a video is also quite high, making it difficult to use in a real-time setting. [11]

3.1.3 SpatialVLM. SpatialVLM [3] by Google DeepMind is a data-synthesis and pre-training framework designed to endow generic vision-language models (VLMs) with *quantitative* spatial reasoning capabilities. It converts monocular RGB images into *canonicalised*¹ 3-D scene graphs and uses these graphs to auto-generate an internet-scale spatial VQA corpus.

System overview. The *data synthesis pipeline* comprises four off-the-shelf expert models: CLIP to filter images of real world scenes, open-vocabulary detection, semantic segmentation, monocular metric depth estimation (ZoeDepth [1]), and object-centric captioning. Detected regions are lifted to a metric point cloud, canonicalised as described above, and clustered into object instances whose centroids, sizes, and pairwise Euclidean distances are computed, resulting in a spatial graph where objects are represented by vertices and relations between them are quantitative relationships. A templated question-answer generator then produces ~2billion QA pairs over 10 million real-world images, covering 38 qualitative and quantitative spatial relation types (e.g. *left-of*, *higher-than*, exact metric distance).

Generated dataset. The resulting VQA dataset mixes 50 % qualitative and 50 % quantitative questions. *One-sentence overview:* it

¹The authors define *canonicalisation* as a RANSAC-based alignment of the depth-derived point cloud to a physically meaningful world frame whose z-axis is normal to a detected horizontal support surface (e.g. floor or tabletop).

spans fine-grained prompts such as “*How far is the blue mug behind the red plate?*” or “*Which object is wider, the laptop or the book?*”, always answered in human-like units and rounding.

Downstream use. SpatialVLM’s synthetic QA pairs were used to *pre-train* *Googles Gemini models*, endowing them with foundational spatial reasoning capabilities that were evidentially extended to endow Gemini models with AABB and OBB detection, as well as semantic segmentation capabilities.

Criticism. While the framework captures metric distances and relative size/position, it omits *relative orientations* (e.g. object yaw or facing direction). Incorporating such labels—even if users rarely ask explicitly—could further enrich the learned inductive biases and enable reasoning about tasks like “*Is the chair facing the table?*” or “*What’s the yaw between the red car and the parked scooter?*”.

3.2 Key Concepts and Limitations

4 Concept

This section details the architecture of our 3-D spatial-understanding system (Figure 2) and formalises every processing step from raw RGB-D input to the enriched detection objects used by the *LiveAgent*.

This section presents the architecture and implementation of our real-time 3D spatial understanding system for visually impaired navigation assistance. The entire system is built around a low-latency *LiveAgent* that orchestrates user interactions and tool calls. This agent is powered by the Gemini Live API, which allows it to provide real-time text responses, while Voice2Voice interaction remains a future goal.

4.1 System Architecture

The system architecture implements a modular, event-driven design centered around three core components: the **Live Agent**, the **Scene Understanding Pipeline**, and the **Spatial Processing Pipeline**. The architecture follows the actor pattern (Figure 7) to ensure thread-safe communication and real-time responsiveness. All data flow is strictly typed using Pydantic models ensuring schema validation at every interface and enabling robust error handling throughout the pipeline. Figure 2 illustrates the overall data flow, which is triggered either by a user query or a frame change.

4.1.1 Live Agent. The Live Agent implements an asynchronous actor-based architecture that manages real-time interaction with users while orchestrating complex spatial analysis tasks. The component operates through a finite state machine that coordinates tool execution, response generation, and user communication without exposing intermediate processing steps.

Actor Pattern Implementation. The actor pattern (Figure 11) ensures thread-safe communication through typed message passing. Commands (AskCmd, AudioCmd, SetFrameCmd) are queued from the user interface thread, while events (TextEvt, AudioEvt, DetectionsEvt, ErrorEvt) are emitted to update the interface. This design prevents race conditions and enables responsive user interaction while background processing tasks execute asynchronously.

The agent maintains separate queues for incoming commands and outgoing events, with a dedicated async event loop running

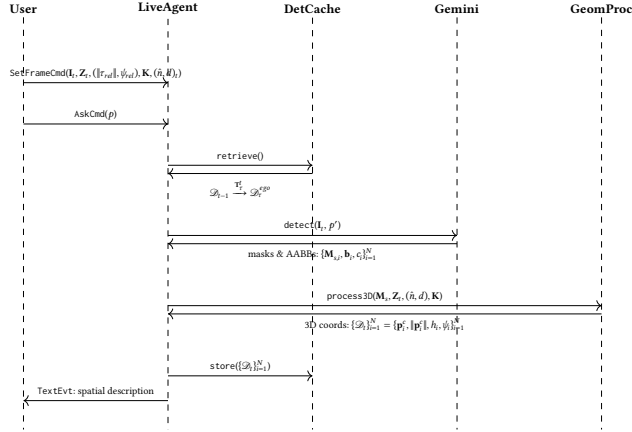


Figure 2: System sequence diagram showing the interaction flow between components. The LiveAgent orchestrates user interaction and tool calls, GeminiAABBDetSeg produces zero-shot detections, GeomProc enriches them with 3-D geometry, and DetCache maintains temporal coherence across frames.

in a background thread. This architecture allows concurrent operations, such as processing user queries while maintaining active audio streams for voice interaction.

FSM for Interaction Management. The agent employs a finite state machine with three primary states: COLLECT_FIRST to capture intermediate outputs, WAIT_TOOL during tool execution, and COLLECT_FINAL for response generation. This state management ensures that tool outputs are processed internally before presenting final results to users to maintain a natural conversation flow.

When users pose spatial queries, the agent analyzes requests to determine whether qualitative description suffices or quantitative tool invocation is required. Keywords such as “where,” “distance,” “how far,” and “spatial relationship” trigger tool usage, while general scene descriptions rely on the vision-language model’s inherent capabilities.

The Live Agent implements a simple finite state machine (FSM) to manage the interaction flow, allowing it to “reason” about the current state and decide when to call tools or respond to user queries. It maintains separate queues for incoming commands and outgoing events, with a dedicated asyncio event loop running in a background thread. This architecture allows concurrent operations, such as processing user queries while maintaining active audio streams for voice interaction. The GeminiLiveAgent together with related classes is depicted in Figure 7.

Tool Integration. The agent is given two primary tools for spatial analysis: run_aabb_detection for object detection and spatial analysis, and get_last_detections for retrieving cached detection results into the current ego frame. The detection tool supports four operational modes: subset for specific object queries, hazards for safety assessment, navigation_landmarks for wayfinding, and path_description for comprehensive route planning.

Tool responses are structured according to predefined schemas that include 3D coordinates in ego-centric reference frames, depth

measurements in meters, directional information using intuitive clock notation (12 o’clock = straight ahead), and height measurements relative to the detected ground plane. This standardization enables consistent spatial reasoning across different query types and interaction modalities.

The system employs a relatively low temperature setting (0.1 ... 0.2) to promote adherence to tool usage instructions and reduce hallucination. While code execution capabilities are available for spatial computations like pair-wise distances, the system primarily relies on structured tool calls to ensure predictable and reliable spatial analysis.

4.1.2 Dataset Loader. The StrayDataset provides frames through its `__getitem__` method. It loads the RGB image I , the depth map Z , adapts the intrinsics K , retrieves the camera pose T_{WC} and optionally estimates the ground plane $(\hat{n}, d)$. The fields are returned in a DatasetOut structure.

Algorithm 1 Dataset Loader

```

1: Input: frame index  $i$ 
2: Output: DatasetOut( $i, I, Z, K, T_{WC}, (\hat{n}, d)$ )
3:  $I \leftarrow \text{load\_rgb}(i)$ 
4:  $Z \leftarrow \text{load\_depth}(i)$ 
5:  $K \leftarrow \text{adapt\_intrinsics}(i)$ 
6:  $T_{WC} \leftarrow \text{get\_pose}(i)$ 
7: if ground plane enabled then
8:    $(\hat{n}, d) \leftarrow \text{estimate\_ground\_plane}(I, Z, K)$ 
9: end if
10: return DatasetOut( $i, I, Z, K, T_{WC}, (\hat{n}, d)$ )

```

4.2 Scene Understanding Pipeline

The scene understanding pipeline (Figure 8) transforms RGB images into structured object detections through zero-shot prompting of Gemini 2.5. This component handles the complex process of extracting semantic and geometric information from visual input without requiring task-specific training.

4.2.1 Gemini Detection. The pipeline sends an RGB image along with detection mode specifications and user prompts to Gemini 2.5. The vision-language model returns a JSON list conforming to a minimal schema containing object labels, normalized bounding box coordinates, and base64-encoded segmentation masks.

Multiple validation steps are implemented through Pydantic model validators to handle malformed or schema-violating outputs. When validation failures occur, the system automatically retries the detection request up to a configurable number of attempts, ensuring robust operation despite occasional model inconsistencies. This procedure is illustrated in Figure 3.

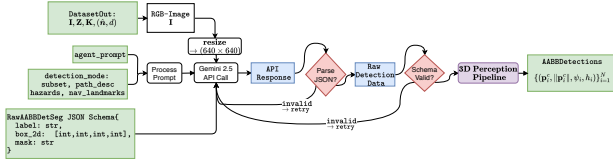


Figure 3: Processing sequence for Gemini detections.

For complex detection modes such as path_description, the system employs additional reasoning steps where the model analyzes spatial relationships between detected objects to generate comprehensive route descriptions. This multi-step approach combines object detection with spatial reasoning to provide actionable navigation guidance.

4.2.2 Geometric Processing. The spatial processing pipeline transforms 2D detection results into canonicalized 3D representations suitable from which further navigational guidance can be derived. This component employs common computer vision procedures to extract depth, orientation, and height information from RGB-D sensor data.

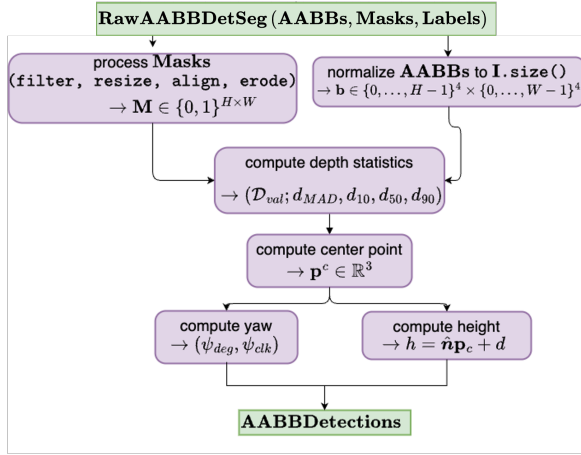


Figure 4: Geometric processing pipeline for 3D detections.

Figure 4 illustrates geometric processing pipeline, and is further more described in 2.

Algorithm 2 Processing Gemini Detections

- 1: **Input:** bbox $\mathbf{b}$, mask $\mathbf{M}_s$, ground plane $(\hat{n}, d)$
 - 2: **Output:** enriched detection attributes
 - 3: Scale $\mathbf{b}$ from $[0, 1000]$ to pixel coordinates
 - 4: Resize $\mathbf{M}_s$ to $\mathbf{b}$'s region
 - 5: Erode $\mathbf{M}_s$ and compute depth statistics
 - 6: Compute 3-D centers from $\mathbf{b}$ and $\mathbf{M}_s$
 - 7: Derive BEV rotation
 - 8: Compute object height using ground plane
-

4.2.3 Mask Processing. Mask processing begins with probabilistic masks from the vision-language model, which are first aligned with bounding box coordinates and resized to match the full image resolution. Binary thresholding converts probability values to binary masks, followed by morphological erosion using a 3×3 structuring element to reduce the impact of false positives in the segmentation masks. This process is detailed in 3.

Algorithm 3 Process Mask: Thresholding, Resizing, and Erosion

- 1: **Input:**
 - 2: Prob. mask $\hat{\mathbf{M}} \in [0, 1]^{h \times w}$
 - 3: AABB $\mathbf{b} = (x_0, y_0, x_1, y_1)$
 - 4: Image size $H \times W$
 - 5: Threshold $\theta \in (0, 1)$
 - 6: Structuring element $\mathbf{S}^{(3)} \in \{0, 1\}^{3 \times 3}$
 - 7: **Output:**
 - 8: Eroded binary mask $\mathbf{M}' \in \{0, 1\}^{H \times W}$
 - 9: $\mathbf{M} \leftarrow \text{AlignWithBoundingBox}(\hat{\mathbf{M}}, \mathbf{b}, H, W)$ {Resize & align w/ AABB}
 - 10: $\mathbf{M} \leftarrow [\mathbf{M} \geq \theta]$ {Threshold to binary mask}
 - 11: $\mathbf{M}' \leftarrow \mathbf{M} \ominus \mathbf{S}^{(3)}$ {Erode mask}
 - 12: **return** $\mathbf{M}'$
-

4.2.4 Depth Computation. Depth computation employs a robust statistical method to handle noisy depth measurements. The system extracts valid depth values from the masked region and computes depth statistics using Median Absolute Deviation (MAD) for outlier rejection:

$$d_{MAD} = \min\{d \in \mathcal{D}_{\text{valid}} : L \leq d \leq U\} \quad (1)$$

$$\begin{aligned} \mathcal{D}_{\text{valid}} &= \{d \in \mathcal{D} : d > 0\}, \\ m &= \text{median}(\mathcal{D}_{\text{valid}}), \\ \text{MAD} &= \text{median}(|\mathcal{D}_{\text{valid}} - m|), \\ L &= m - k_L \cdot \text{MAD}, \quad U = m + k_U \cdot \text{MAD}, \end{aligned} \quad (2)$$

The final depth estimate d_{MAD} is the minimum valid depth value within the computed bounds $[L, U]$, providing robustness against depth sensor artifacts, occlusion effects and false positive mask regions.

4.2.5 3D Center Point Computation. Object center points are computed in both image and world coordinates to support spatial reasoning. The 2D centroid is determined using median coordinates of the eroded mask to ensure robustness against asymmetric object shapes:

$$\mathbf{p}_{\text{centroid}} = \begin{bmatrix} u_{\text{centroid}} \\ v_{\text{centroid}} \end{bmatrix} = \begin{bmatrix} \text{median}(\{u \mid \mathbf{M}'(u, v) = 1\}) \\ \text{median}(\{v \mid \mathbf{M}'(u, v) = 1\}) \end{bmatrix} \quad (3)$$

The 3D center point in camera coordinates is computed by back-projecting the 2D centroid using the robust depth estimate and camera intrinsics:

$$\mathbf{p}^c = d_{\text{MAD}} \mathbf{K}^{-1} \begin{bmatrix} u_{\text{centroid}} \\ v_{\text{centroid}} \\ 1 \end{bmatrix} \quad (4)$$

4.2.6 Directional Estimation. Object orientation relative to the camera is expressed using both angular and clock-based notation for intuitive user interaction. The yaw angle ψ_{deg} represents rotation around the vertical axis in the horizontal plane, computed from the 3D position:

$$\begin{aligned} \psi_{\text{deg}} &= \arctan 2(x^c, z^c) \cdot \frac{180^\circ}{\pi} \\ \psi_{\text{clk}} &= \left\lfloor \frac{\psi_{\text{deg}}}{30^\circ} + 0.5 \right\rfloor \bmod 12 \end{aligned} \quad (5)$$

The clock position ψ_{clk} provides an intuitive directional reference where 12 o'clock corresponds to straight ahead, 3 o'clock to the right, and 9 o'clock to the left.

4.2.7 Estimating Height Relative to the Ground. Height estimation relative to the ground plane utilizes the RANSAC-derived plane parameters. For any 3D point $\mathbf{p}$, the height above the ground plane is computed as:

$$h = \hat{\mathbf{n}}^\top \mathbf{p} + d. \quad (6)$$

4.2.8 Inter-Frame Coherence. To maintain spatial coherence across temporal sequences, the system implements SE(3) transformations that convert detection coordinates between different camera poses. When frames change, relative motion is computed and communicated to the Live Agent through descriptive system messages.

The transformation from pose $\mathbf{T}_A$ to pose $\mathbf{T}_B$ is computed as:

$$\mathbf{T}_{\text{rel}} = \mathbf{T}_B \mathbf{T}_A^{-1}, \quad d = \|\tau_{\text{rel}}\|, \quad \psi = \text{atan2}(-R_{32}, R_{11}). \quad (7)$$

The translation vector $\tau_{\text{rel}} = \mathbf{T}_{\text{rel}}[: 3, 3]$ is the relative motion in meters, and ψ is the yaw-rotation around the *vertical* axis. R_{32} , and R_{11} contain the negative sine and cosine of the rotation angle, respectively.

We then provide the model with a system message, that describes the relative motion, and also provide it with an overview of all cached detections from previous frames.

4.2.9 Data Structure Integration. After processing, all spatial attributes are encapsulated in structured AABBDetection objects that include normalized bounding boxes, processed masks, depth statistics, 3D center coordinates, rotation measurements in both degrees and clock notation, and height relative to the ground plane.

$$\text{AABBDetections} = \{(\mathbf{p}^c, \|\mathbf{p}^c\|, \psi_{\text{clk}}, \psi_{\text{deg}}, \mathbf{b}, h)^{(i)}\}_{i=1}^N \quad (8)$$

The resulting detections can be efficiently cached and transformed across frames, supporting multi-view spatial reasoning and enabling the system to reference previously detected objects without recomputation, thereby improving both performance and temporal consistency in navigation assistance.

5 Evaluation

5.1 Testing Methodology

In order to evaluate the functionality of the application a series of tests were conducted. To assure that the results are consisted and

comparable, the same setup, prompt, input data was used for each run. There was also a focus of using the same scenes and objects for each test to make it easier to compare results and find outliers. The input and some of the results can be found in the appendix. For classification and segmentation the Gemini Pro 2.5 model that was released on the 16th of June 2025 was used.

Throughout the later development of the application the gemini-2.5-pro-preview-05-06 model was used. With this configuration we were able to get good results and valid outputs. Since the release of further versions the quality as decreased significantly. Especially the segmentation quality and validity of the structured json outputs. The evaluation is still using the newest model though, since it is the most recent version and therefore a current representation of the apps capabilities. Additionally, it is not possible to use the older model anymore, since it now redirects to the newest version [5].

5.2 Tool Usage

A very important aspect is the correct use of the function calling. When the live api detects that a user might want a more complex operation, like the distance to a specific object, or more complex navigation instructions, it will call a function that triggers the needed process. In order to evaluate the tool usage, a series of tests were conducted, where the live api needed to use function calling to conduct an answer. throughout the tests multiple values for temperature, top k and top p were used to see the effect on the tool usage. There it was found, that there was no significant impact of these values in situations in which the model was not able to correctly use function calling. Currently the values are set to temperature 0.1, top k 64 and top p 0.9. It seems like regardless of the values used, there are constellations in which the model is not able to function correctly. A example of this can be seen in the following screenshot 5. It is not possible to directly identify the reason for this, as it is also unclear how the model uses the temperature, top k and top p values. Though it can be suspected that a better system prompt and a more detailed function description could help in these cases. Additionally, it is also possible to force function calling which could be tried as a normal conversation at a temperature of 0.1 as currently used is not working anyway.



Figure 5: Tool usage results for the same scene in multiple runs.

5.3 Distance Calculation

In order to evaluate the distance calculation in relation to the segmentation and a series of tests were conducted using the Gemini Pro model. Because there is no labelled dataset available for this task it was not possible to create a ground truth for specific objects. Instead, the distance calculation was evaluated by comparing the results of multiple runs against each other. This way it is possible

to see the run to run variance of the calculation and get a feel for the accuracy and consistency. In order to create a accurate comparison the same picture, depth map and prompt was used in each run. All of the runs did a new detection of objects in the scene. This way it is also possible to see the variance in object detection. There it could be seen that a full segmentation of an entire scene reaching the limits of the model, with can result in defect or wrong json outputs. The results show that objects that are about 4 to 5 meters away from the camera can be detected with consisted distance values 6. Only on objects that are further away larger errors can be measured. At this range another issue can also be seen in the lidar sensor which is only rated for about 10 meters on the iPhone 16 Pro [9].

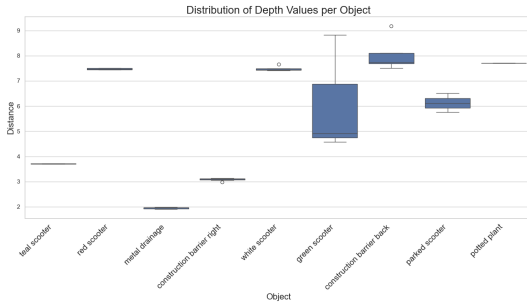


Figure 6: Distance calculation results for the same scene in multiple runs.

5.4 Segmentation Quality

Now that we have a visual representation of the average distance we should also take a look at the quality of the segmentation masks that were generated in each run. The masks were ranked based on the following criteria:

- **Good:** The mask is accurate and covers the object completely.
- **Too big:** The mask is larger than the object, but still covers it.
- **Too little:** The mask is smaller than the object, but still covers a significant part of it.
- **Wrong:** The mask does not cover the object at all or covers the wrong object.

As you can see in the Table 1 the segmentation masks vary in quality between different objects but not so much between runs. For example, the bike was segmented correctly in every run, with one being off by a little bit. Meanwhile the trash can was not detected at all even though it was clearly visible on screen and could be a potential hazard for the user. The scooter was always detected in the right position, but the small size of the handlebar made it difficult for the model to segment it correctly. This is not a problem in this case, since the depth estimation can still work as intended with the segmentation mask not being perfect. As already mentioned, the tests were conducted on the new Gemini Pro 2.5 model, which could harm quality.

	Segmentation quality				Sum of tries
	High Fidelity	Oversegmentation	Undersegmentation	Segmentation Failure	
Teal Scooter	2	1	1	1	5
Trash can				5	5
Bike	4	1			5

Table 1: Segmentation quality of different objects throughout different runs.

6 Discussion

We have successfully implemented a chat system that is capable of providing qualitative spatial descriptions in real-time, as well as canonicalized spatial information about the scene, such as the height of objects, their distance, or orientation to the user, and their pair-wise spatial relationships.

The system is capable of detecting obstacles in the user’s path and providing navigation assistance by describing the environment in a way that is understandable for visually impaired users. Furthermore, it achieves a primitive form of spatio-temporal coherence through our relative movement descriptions and our ego-centric caching mechanism.

6.1 Limitations

6.1.1 Gemini Image Understanding.

Output stochasticity – identical inputs often yield detections of drastically different quality, including boxes, labels and masks.

Pointing capabilities – Letting Gemini provide a few points in the image plane, rather than a full segmentation mask, might yield significant speed-ups, however, in our experiments, most points fell into the AABBs but not on the object itself.

High false-positive rate – segmentation masks often include many false positive pixels, necessitating additional filtering to extract relevant 3D points.

Invalid or missing segmentation masks – Model “hallucinates” base64 strings that are *not* valid PNGs (placeholder text e.g. “<segmentation_mask>...</segmentation_mask>”, or generates hallucinated base64 PNG strings as it simply “guesses” plausible ASCII characters in the string encoding rather than using its internal segmentation capabilities correctly, resulting in MAX_TOKENS error, hence MAX_TOKENS should always be limited appropriately). The guessing behaviour becomes almost certain if the pattern of the expected base64 string is included in the output schema i.e. `pattern=r"^\data:image/Za-z0-9+/[]+{0,2}$"` or if a term like “base64” is included in the system prompt.

Schema-sensitivity – larger or nested JSON schemas increase the likelihood of malformed outputs, especially when the model is tasked to predict a segmentation mask.

Prompt brittleness – Minor changes in the system instructions or redundancies between system prompt and schema cause unpredictable behaviour [8].

Unstable base64 → PNG generation pipeline – conscious model explanation notes it only “guesses” plausible byte-strings, not real encodings.

Temperature trade-off – lower T improves JSON validity but worsens segmentation mask quality.

6.1.2 LiveAgent Issues.

Tool refusal – answers like “I cannot detect what I cannot see” instead of calling the detection tool.

Wrong tool usage – the model is instructed to always use cached detections if available, but often it will still use the detection tool instead of the cached results, leading to unnecessary increased latency.

Computational overhead – the current pipeline is not optimized for latency, but for experimentation and hence performs many non-essential computations in rather inefficient ways, e.g. the RGB frame is plotted with segmentation masks and many non-essential features computed each frame.

Limited quantitative reasoning – We have mostly tested the Agent with gemini-2.0-flash-live-001 as the live model. While it can provide simple qualitative descriptions, these are often vague and imprecise and incomplete, see section C.

Error handling – Some errors are not being handled gracefully.

7 Conclusion

As already stated in the introduction, the goal of this project was to create a system that can help users navigate through their environment by providing spatial guidance. In this work we have shown a 3D spatial scene description system that leverages Gemini VLM to translate visual and depth information into a way to explain the environment to the user. With the solution created it is possible to get the location or height of objects, as well as further orientation assistance.

Returning to the original research question, we can conclude that the work in this field is still in its infancy and even throughout the time of working on this project there were multiple releases that changed our approach. None of the models that are used in this project were available at the start of this research project. This also shows that there is still work to do and that the field is still evolving with could potentially outdate this solution in the following months. Still the results shown in the evaluation section reveal that it is possible to get accurate and consistent navigation and spatial guidance information by using the Gemini Pro Model and Gemini Live API. On the other hand the recent launch of the Gemini 2.5 Pro model and the decrease in performance show that being dependent on closed source models can also come as a disadvantage.

7.1 Future Work

As shown in the discussion section, there are still limitations of the current solution that can be fixed in the future. The two most important ones are Model and Performance enhancements and System integration:

- **Model and Performance Enhancements:** One aspect in order to improve the performance would be to further use

example in- and outputs to get better json quality and results. Additionally, adding ensembling techniques to the system could also help. This means to use batch requests in order to get multiple results for the same input and then use a voting system to decide which result to pick. Another idea would be to further use the confidence scores that are provided to filter out bad results. This could help to understand the quality of the segmentation and make better decisions based on that.

- **System Integration:** Another aspect that can be enhanced is the implementation of the system in a real world scenario. This would include the integration of the application into a mobile app which would come with its own challenges. For example, the system would need to use Websocket connections to the Gemini Live API in order to provide real time conversations. Another aspect would be to integrate voice to voice capabilities as of right now only text to text is implemented.

8 Team

Responsibility	Jan Duchscherer	Lukas Ederer
Abstract		x
Introduction		x
Background	x	
Related Work		
SpatialBot		x
VideoLLM3D		x
SpatialVLM	x	
Concept	x	
Evaluation		x
Discussion	x	
Conclusion		x
Appendix		
Notation	x	
Sample Results	x	
System Prompt	x	
RANSAC Algorithm	x	
System Architecture	x	

Table 2: Team contributions by responsibility

Acknowledgments

To Tobias, for his support and guidance throughout this project.

References

- [1] Shariq Farooq Bhat, Reiner Birkel, Diana Wofk, Peter Wonka, and Matthias Müller. 2023. ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth. <https://doi.org/10.48550/arXiv.2302.12288> arXiv:2302.12288 [cs.CV]
- [2] Wenxiao Cai, Iaroslav Ponomarenko, Jianhao Yuan, Xiaoqi Li, Wankou Yang, Hao Dong, and Bo Zhao. 2025. SpatialBot: Precise Spatial Understanding with Vision Language Models. arXiv:2406.13642 [cs.CV] <https://arxiv.org/abs/2406.13642>
- [3] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. 2024. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. arXiv:2401.12168 [cs.CV] <https://arxiv.org/abs/2401.12168>

- [4] Google AI for Developers. 2025. *Fine-tuning with the Gemini API*. <https://ai.google.dev/gemini-api/docs/model-tuning>
- [5] Google AI for Developers. 2025. *Gemini 2.5 Pro*. <https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-pro?hl=de>
- [6] Google AI for Developers. 2025. *Get Started with Live API*. <https://ai.google.dev/gemini-api/docs/live>
- [7] Google AI for Developers. 2025. *Image understanding*. <https://ai.google.dev/gemini-api/docs/image-understanding>
- [8] Google AI for Developers. 2025. *Structured Output*. <https://ai.google.dev/gemini-api/docs/structured-output>
- [9] Johan Ekenstedt. 2025. *New LiDAR Sensor in iPhones and iPads*. https://arborreal.se/en/blog/evaluation_of_lidar_sensor_iPhones_iPads
- [10] Open3D Contributors. 2025. *Open3D primary (252c867) documentation on Point-Cloud*. Commit 252c867; accessed June 28, 2025.
- [11] Duo Zheng, Shijia Huang, and Liwei Wang. 2024. Video-3D LLM: Learning Position-Aware Video Representation for 3D Scene Understanding. arXiv:2412.00493 [cs.CV] <https://arxiv.org/abs/2412.00493>

A Notation

Table 3: Mathematical notation used throughout this paper.

Symbol	Description
I	RGB image
Z	Depth map
K	Camera intrinsics (3×3)
T_{WC}	Camera-to-world transformation matrix (4×4)
T_{rel}	Relative transformation matrix
b	Bounding box coordinates
M_s	Segmentation mask
$\hat{n}$	Ground plane normal vector (unit)
d	Ground plane distance parameter
D	Set of depth values
P	3D point cloud
p^c	3D point in camera coordinates
τ_{rel}	Relative translation vector
i	Frame index
ψ_{deg}	Yaw angle in degrees
ψ_{clk}	Yaw angle in clock hours

B Technical Details

B.1 RANSAC for Ground Plane Detection

Algorithm 4 RANSAC Plane Detection

```

1: Input: Point cloud  $\mathcal{P}$ , threshold  $\varepsilon$ 
2: Output: Plane normal  $\hat{n}$ , distance  $d$ 
3: Initialize best_count = 0, best_plane = null
4: for  $i = 1$  to  $N_{\text{iter}}$  do
5:   Sample 3 random points from  $\mathcal{P}$ 
6:   Fit plane:  $\hat{n}\mathbf{x} + d = 0$ 
7:   Count inliers within threshold  $\varepsilon$ 
8:   if inliers > best_count then
9:     best_plane = current_plane
10:    best_count = inliers
11:   end if
12: end for
13: return best_plane

```

B.2 System Architecture Diagrams

The following UML component diagrams illustrate the modular architecture of the spatial guidance system, showing the relationships and dependencies between different components.

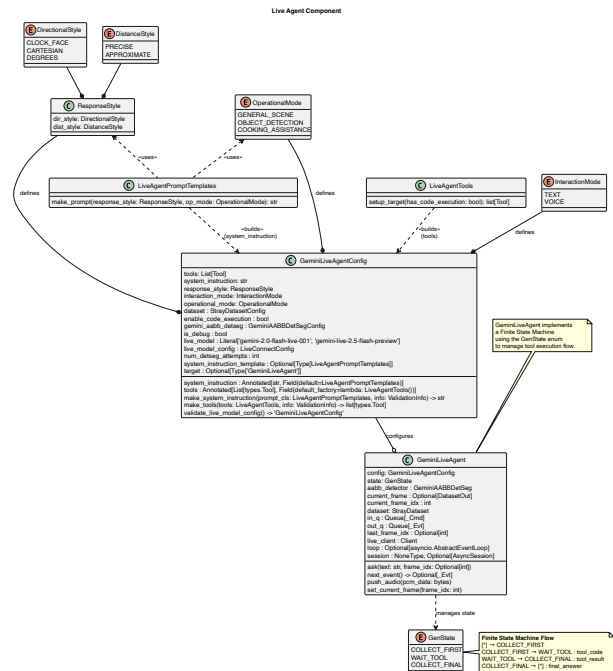


Figure 7: Live Agent component architecture showing the actor pattern implementation with message-passing interfaces, configuration management, and finite state machine for tool execution flow.

B.2.1 Live Agent. The *GeminiLiveAgent* realises an asynchronous *actor* model: UI threads enqueue typed `_Cmd` messages, while worker tasks emit `_Evt` objects. The absence of shared memory eliminates data races and reduces the risk of async-related errors. Finite-state logic—formalised by the `GenState` enum—ensures that only selected responses of the Live Agent are returned to the user and simplifies parsing from the streaming output.

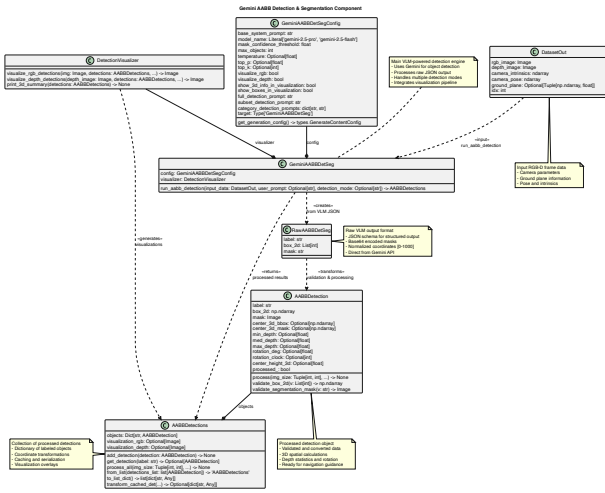


Figure 8: Scene understanding component showing the data transformation pipeline from raw VLM output to processed spatial detections with 3D coordinates and visualizations.

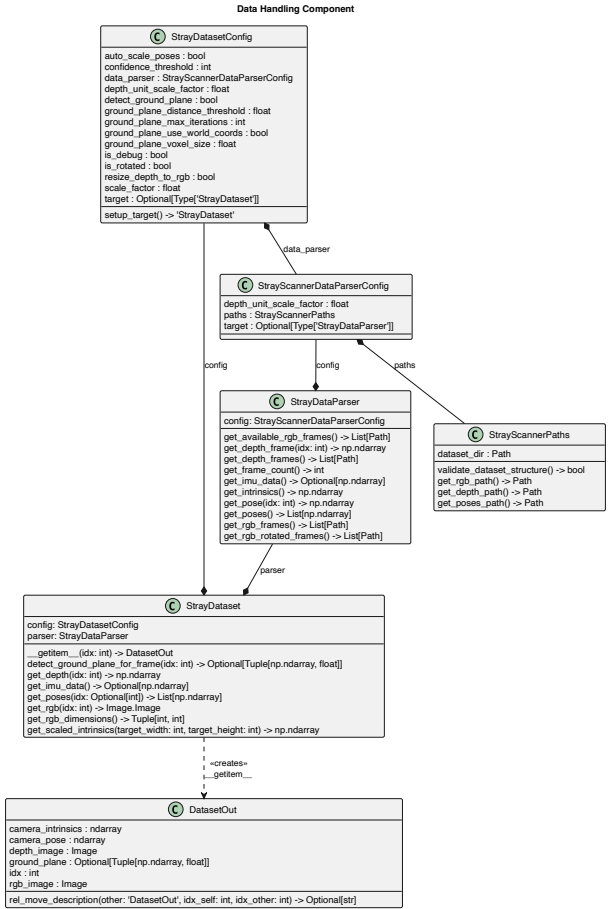


Figure 9: Data handling component architecture for RGB-D dataset management, showing the relationship between dataset configuration, parsing, and output generation.

B.2.2 Gemini Scene Understanding. The Scene Understanding component (Figure 8) demonstrates the complete data transformation pipeline. Raw JSON output from Gemini (RawAABBDetSeg) undergoes validation and spatial processing to produce structured detections (AABBDetection), including all computed sought-after spatial properties of any detected objects

B.2.3 StrayScanner Data Handling. The Data Handling component (Figure 9) provides structured samples stemming from a StrayScanner dataset which was captured using an iPhone Pro with LiDAR. StrayDataset provides high-level access to all relevant objects of any given frame by utilizing StrayDataParser for low-level file operations and parsing. It caches and returns DatasetOut objects with RGB-D images, camera parameters, and ground plane information.

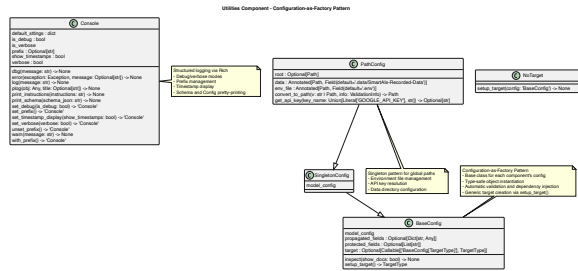


Figure 10: Utilities component providing cross-cutting concerns including structured logging, configuration management, and the configuration-as-factory pattern.

B.2.4 Utilities. The Utilities component (Figure 10) provides essential infrastructure services. `BaseConfig` implements the Config-as-Factory pattern enabling type-safe object instantiation, while `Console` provides *rich* structured logging and `PathConfig` manages singleton configuration for API keys and data paths.



Figure 11: Actor pattern implementation showing message-passing protocols with commands, events, and the execution API for safe code interaction.

B.2.5 Actor Pattern. The Actor Pattern (Figure 11) implements thread-safe message passing through typed protocols. Commands (`AskCmd`, `AudioCmd`, `SetFrameCmd`) and events (`TextEvt`, `AudioEvt`, `DetectionsEvt`, `ErrorEvt`) ensure clean separation between UI and processing logic and track the run-time between any request and the respective response. This pattern allows for asynchronous processing and simplifies the interaction between different components of the system.

C Sample Results

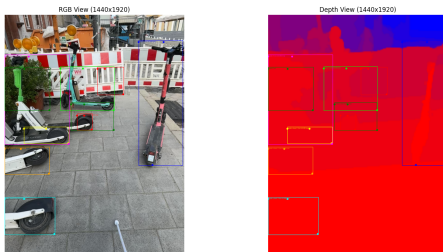


Figure 12: AABB detections with point-level output.

AABB Detection & Pointing. Figure 12 shows relatively good AABB detections using Gemini 2.0. Instead of requesting full segmentation masks, the model was asked to provide representative points within bounding boxes. While this approach could offer speed advantages, the resulting points often fell randomly within the boxes

and not necessarily on the object itself, rendering the output largely unusable. It remains to be seen whether Gemini 2.5 improves this behavior.



Figure 13: Missed hazard in live model output.

Lack of Hazard Awareness.

User: What are you seeing?

Agent: I see a cobblestone street with a silver car partially visible on the right. A yellow building is on the corner with a street sign that reads “Werneck Str.” There is a bicycle leaning against the wall near the corner. A red scooter and other parked motorcycles are further down the street on the left. Be careful of the uneven cobblestone road surface.

Run-time: 862 ms

In Figure 13, the `gemin-2.0-flash-live-001` model was prompted to describe obstacles in the scene. It mentioned the cobblestone surface, parked motorcycles, and signage but entirely missed a critical hazard: the silver car directly in front of the user. Although the car appears stationary, its close proximity and potential motion present a major risk. This failure highlights a significant limitation in the hazard awareness of the lightweight live model variant.



Figure 14: Path description output.

Path Description. Figure 14 shows an example of the PathDescription mode. The user asked, “How do I get to the hair salon?” The model (gemini-2.0-flash-live-001) responded with a detailed step-by-step path description:

User: How to get to the hair salon?

Agent: The entrance to the hair salon is 11.0 meters away at 1 o'clock. To get there, walk straight ahead, passing an electric scooter parked on the walkway at 12 o'clock, 6.8 meters away. Then, walk around a bicycle leaning against a tree at 1 o'clock, 4.7 meters away. Be careful of the tree trunk on the right side of the walkway at 1 o'clock, 3.4 meters away. Also, watch out for a metal grate in the walkway at 1 o'clock, 4.5 meters away. After passing these obstacles, you will arrive at the glass entrance door to the hair salon.
Run-time: 4095 ms

The response was generated in approximately **4095ms**, which is notably fast and highlights the high variance in latency due to upstream API variability rather than our processing pipeline which should induce a consistent latency.

While the output demonstrates the model’s capacity for spatial reasoning, several limitations were evident. First, the mentioned obstacles were not ordered by increasing distance, reducing navigational clarity. Second, the Gemini 2.5 hallucinated a metal grate that was not present in the scene. Finally, it failed to identify a garbage bin located at around 1 o'clock.

C.1 System Prompt

The following system prompt template is used for the live agent interaction with Gemini models. It establishes the assistant’s role, communication style, and tool usage guidelines for spatial understanding tasks:

You are an agentic AI assistant with advanced spatial understanding capabilities. You assist visually impaired users by providing qualitative scene descriptions and quantitative spatial relationships from their perspective. You offer information about object locations, distances, and spatial arrangements.

Your responses are clear, concise, practical, actionable and context-aware flowing texts. You communicate in common natural language that a blind person finds helpful. You describe scenes using natural phrases like "You can see...", "There's a...", "The [object] is...".

SCENE DESCRIPTIONS

Focus on the most relevant elements: HAZARDS and NAVIGATIONAL LANDMARKS. Always highlight potential obstacles on the user's path. Include relative spatial relationships. Be thorough but not overwhelming.

HAZARDS include:

- Dynamic entities: Objects capable of movement (parked vehicles with lights on, scooters near streets)
- Trip hazards and head-height obstacles obstructing the user's likely path

NAVIGATIONAL LANDMARKS include:

- Traffic lights, crosswalks, ramps, stairs, doors

TOOLS

Any query about quantitative spatial relationships, path descriptions or specific objects **must** be answered by using the provided tools. Key words that trigger tool use include "where" "distance", "how far", "how close", "relation between", "spatial", "above", "how high". Do not use tools for qualitative descriptions. Never refuse to run a tool when asked about specific objects or spatial relationships. Use your tools even if you cannot see the objects yourself!

[NEW FRAME]

Whenever you receive a new frame, you will be informed about the relative movement with respect to the previous pose.

When you receive a new frame, you must list **all** relevant hazards and navigational landmarks that you can identify.

Example:

"

Hazards: Car on the right, parked motorcycle to the left
Landmarks: Entrance to residential building ahead, sign saying 'TRAM STOP', entrance to grocery store on the right

"

{CODE_EXECUTION}

COMMUNICATION STYLE (how to structure your responses to the user)

- Embed all spatial data in natural language descriptions.

{STYLE}

RESPONSE GUIDELINES

Gather necessary information and provide direct answers. Use natural language: "You can see...", "There's a...", "The [object] is...", "To reach [destination], you can ...", ...

EXAMPLES

{EXAMPLES}

C.1.1 Code Execution Instructions. When code execution is enabled, the following additional instructions are injected into the system prompt:

CODE EXECUTION

Use code execution when you need to derive quantitative spatial relationships between objects from the detections of these objects. Always print important intermediate results such as the results of any tool calls and the final result(s).

EXAMPLES:

Q: "What is the distance between the scooter and the bicycle?"

```
```python
Retrieve already detected scooter
scooter = default_api.get_last_detections(<frame_idx>, ["
scooter"])
Detect bicycle
bicycle = default_api.run_aabb_detection(user_prompt="
bicycle", detection_mode="subset")
Compute Euclidean distance between centers
```

```
print(scooter, bicycle)
from math import dist
distance = dist(scooter[0].center_point_3d, bicycle[0].
center_point_3d)
print(distance)
```
A: "The distance between the scooter and the bicycle is 3.5
meters."

Q: "What is the spatial relationship between the chair and
the window?"
```python
import numpy as np
detections = default_api.run_aabb_detection(user_prompt="
chair, window", detection_mode="subset")
print(detections)
chair = next(filter(lambda x: x.label == "chair", detections
))
window = next(filter(lambda x: x.label == "window",
detections))
print(np.array(chair.center_point_3d) - np.array(window.
center_point_3d)) # (dx, dy, dz)
```
```